STOK AÍ: DESENVOLVIMENTO DE UM SISTEMA WEB MULTIEMPRESA PARA GESTÃO DE ESTOQUES COM OPERAÇÃO POR WHATSAPP ASSISTIDA POR INTELIGÊNCIA ARTIFICIAL

Samuel Alves Luiz[1]; Jonatas Lopes Vidal[1]; Pietro[1]

ORIENTAÇÃO
Orientador(a): a definir[1]
1 IFRN - Campus Nova Cruz.

RESUMO

A prática profissional consistiu no desenvolvimento do Stok Aí, um sistema web multiempresa de gestão de estoques voltado a pequenas e médias operações, originado da demanda real de uma empresa que controlava o almoxarifado em planilhas. O objetivo central foi transformar um controle rígido, com dois depósitos fixos escritos no próprio código, em um produto configurável, no qual unidades, estoques e modos de operação são cadastro e não programa. A metodologia foi incremental: migrações numeradas e reversíveis para evoluir o banco de dados, a regra de saldo isolada em funções puras cobertas por testes automatizados, isolamento de dados por empresa fiscalizado pelo próprio banco e operação pelo WhatsApp na qual a inteligência artificial interpreta a mensagem e o código executa a movimentação. Resultaram cinco modos de estoque, 57 migrações, 635 testes automatizados e um sistema em produção, com painel, alertas, auditoria, recontagem e cotações a fornecedores.

PALAVRAS-CHAVE: Gestão de estoques, sistema web, multiempresa, WhatsApp, inteligência artificial.

ABSTRACT

This professional practice consisted of developing Stok Ai, a multi-tenant web system for inventory management aimed at small and medium operations, born from the real demand of a company that controlled its warehouse using spreadsheets. The central objective was to turn a rigid control, with two fixed warehouses written into the code itself, into a configurable product in which units, stocks and operating modes are registry data rather than program. The methodology was incremental: numbered and reversible migrations to evolve the database, the balance rule isolated in pure functions covered by automated tests, per-company data isolation enforced by the database itself, and operation through WhatsApp in which artificial intelligence interprets the message while code executes the movement. The results were five stock modes, 57 migrations, 635 automated tests and a system in production, with dashboard, alerts, audit, recount and supplier quotations.

KEYWORDS: Inventory management, web system, multi-tenant, WhatsApp, artificial intelligence.

1 INTRODUÇÃO

Estoque é dinheiro parado. Cada saco de cimento, cada luva e cada caixa de parafusos guardados representam capital que foi desembolsado e ainda não se transformou em obra entregue ou serviço prestado. Faltar material interrompe a produção; sobrar material imobiliza recursos, ocupa espaço e, em itens perecíveis, vira perda. A administração de materiais existe precisamente para equilibrar esses dois riscos, e para isso depende de uma informação simples que, na prática, quase nunca está disponível: quanto existe, de cada item, em cada lugar, agora (DIAS, 2019; BALLOU, 2006).

Em grandes empresas essa informação é responsabilidade de sistemas integrados de gestão. Em pequenas e médias operações, no entanto, o controle costuma viver em cadernos, em planilhas compartilhadas por aplicativos de mensagem ou na memória do almoxarife. O resultado é previsível: saldos que ninguém confia, compras feitas por pressentimento, material comprado duas vezes porque estava guardado em outro depósito e nenhuma forma de responder quem retirou o quê, quando e para qual finalidade. A ausência não é de disciplina, mas de ferramenta: os sistemas disponíveis no mercado são, em geral, caros, pensados para a indústria e exigem que a equipe pare de trabalhar para alimentar o cadastro.

O projeto aqui relatado nasceu desse cenário concreto. Uma empresa de construção civil e administração de imóveis do interior do Rio Grande do Norte mantinha dois almoxarifados — um central, de onde o material chega e é guardado, e um de obra, onde o material é consumido — e controlava ambos em planilhas. A primeira versão do sistema desenvolvido pela equipe atendeu a essa realidade literalmente: a tabela de materiais tinha duas colunas de saldo, chamadas saldo_central e saldo_obra. Enquanto o saldo era coluna, "Central" e "Obra" não eram cadastro da empresa, eram conceito do programa. Abrir uma unidade em Natal, criar um terceiro depósito ou separar o estoque de equipamentos de proteção individual do estoque de insumos exigiria alterar o código-fonte, e não preencher um formulário.

Esse limite revelou o problema de pesquisa que orientou a prática profissional: como evoluir um sistema que já está em uso, com dados reais dentro dele, para que a estrutura do estoque deixe de ser decidida pelo programador e passe a ser decidida pela empresa, sem interromper a operação e sem perder a confiança no saldo? A pergunta é de engenharia de software tanto quanto de gestão de materiais, e é ela que liga o trabalho às áreas temáticas de desenvolvimento de sistemas, banco de dados e segurança da informação do curso técnico em Informática.

O objetivo geral foi desenvolver o Stok Aí: um sistema web multiempresa para gestão de estoques, configurável pela própria empresa usuária, capaz de registrar e auditar movimentações, avisar quando um material está acabando e permitir que a equipe opere pelo celular, por meio do WhatsApp, sem precisar de aplicativo instalado.

Como objetivos específicos, delimitaram-se: a) substituir o saldo em colunas fixas por saldo por depósito, transformando unidade e estoque em cadastro, sem indisponibilidade do sistema e sem perda de histórico; b) isolar a regra que decide se uma movimentação pode ocorrer em funções puras, cobertas por testes automatizados; c) garantir que os dados de uma empresa não possam ser lidos por outra, com a verificação feita pelo banco de dados e não apenas pelo código da aplicação; d) implementar modos de operação distintos para o mesmo sistema — almoxarifado, empréstimo, entrega, controle de validade e patrimônio; e) oferecer operação por WhatsApp com interpretação da mensagem por um modelo de linguagem, mantendo a execução da movimentação sob responsabilidade do código; e f) disponibilizar painel, alertas de estoque mínimo, auditoria, recontagem e cotação a fornecedores.

O público beneficiário direto é a equipe de operação das empresas usuárias: almoxarifes, encarregados de obra, administrativo e proprietários — pessoas que, em geral, não têm formação em

informática e usam o sistema pelo celular, muitas vezes em pé, no canteiro de obras. Quantitativamente, a estrutura atual atende empresas com até quatro unidades, com depósitos e usuários ilimitados no plano superior, e já opera com organizações reais em produção. Qualitativamente, o benefício esperado é a substituição do controle por memória e planilha por um registro rastreável, com responsável, data e justificativa em cada movimentação. Para os discentes, o benefício é a experiência de manter um software em produção, com usuários que reclamam quando algo quebra — uma condição que nenhum exercício de sala de aula reproduz.

Os principais desafios antevistos foram três. O primeiro, de ordem técnica, era migrar o modelo de dados com o sistema no ar: um erro de ordem entre a publicação do código novo e a execução da migração no banco faria o saldo "envelhecer em silêncio", divergindo da realidade sem mensagem de erro. O segundo, de ordem arquitetural, era impedir o vazamento de dados entre empresas em um banco compartilhado, com cerca de uma centena de pontos de consulta no servidor. O terceiro, de ordem metodológica, era delimitar o papel da inteligência artificial na operação: permitir que ela interprete linguagem natural sem permitir que ela decida sozinha mexer no estoque de alguém.

O artigo está organizado em cinco seções. A seção 2 reúne os conceitos de gestão de estoques, arquitetura de aplicações web, multiempresa e interfaces conversacionais que fundamentaram as decisões. A seção 3 descreve a metodologia, as tecnologias e o processo de desenvolvimento. A seção 4 apresenta e discute os resultados, incluindo um defeito de concorrência observado em produção e o que ele ensinou. A seção 5 traz as considerações finais, as limitações do trabalho e os encaminhamentos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Gestão de estoques: ponto de recompra, ruptura e inventário rotativo

A administração de materiais trata o estoque como um amortecedor entre o fornecimento e o consumo, e mede sua eficiência pelo equilíbrio entre o custo de manter e o custo de faltar (DIAS, 2019). Dois conceitos dessa literatura foram transportados diretamente para o sistema. O primeiro é o estoque mínimo, ou ponto de recompra: a quantidade a partir da qual é preciso comprar, para que o material não acabe antes de a reposição chegar. O segundo é a ruptura: a situação em que o saldo chega a zero e a operação para. Ballou (2006) e Chopra e Meindl (2016) destacam que a utilidade do ponto de recompra depende da unidade de análise escolhida: um saldo agregado pode esconder a falta em um ponto específico da cadeia.

Essa observação teve consequência prática direta no projeto. Decidiu-se que o estoque mínimo é o ponto de recompra da unidade, e não de cada depósito isolado: faltar material em um dos depósitos de Nova Cruz enquanto sobra no outro é um problema de transferência interna, não de compra; faltar em Natal enquanto sobra em Nova Cruz, por outro lado, é compra em Natal, porque unidades distintas são estoques distintos. O sistema, por isso, distingue cinco situações para um material — ruptura, transferir, abaixo do mínimo, atenção e normal — em vez das duas usuais (falta ou não falta), já que cada situação implica uma ação administrativa diferente.

Um terceiro conceito incorporado foi o do inventário rotativo: a conferência física periódica de parcelas do estoque, em vez de uma contagem geral anual (DIAS, 2019). O que se conta não é apenas a divergência encontrada, mas o fato de a contagem ter passado por aquele item — informação necessária para responder o que ainda falta conferir. Por fim, para materiais perecíveis adotou-se a regra FEFO (first expired, first out), em que a saída consome primeiro o lote de validade mais próxima, prática recomendada para reduzir perdas por vencimento (BALLOU, 2006).

2.2 Arquitetura de aplicações web e separação de responsabilidades

O sistema foi construído como uma aplicação de página única (SPA) que consome uma interface de programação de aplicações (API) sobre o protocolo HTTP, arranjo hoje predominante em sistemas de gestão acessados pelo navegador. Fowler (2003) descreve os padrões recorrentes dessa separação, enquanto Martin (2019) sustenta um princípio que orientou a organização do código-fonte: as regras de negócio devem ocupar o centro do sistema e não depender de infraestrutura — banco de dados, rede ou interface são detalhes que ficam nas bordas.

A aplicação prática desse princípio no projeto consistiu em escrever a regra que decide se uma movimentação de estoque pode ocorrer como função pura: recebe os saldos atuais e a operação pretendida, devolve os saldos resultantes ou a recusa justificada, sem consultar banco e sem produzir efeito algum. Funções com essa característica podem ser testadas isoladamente, de forma rápida e determinística, o que viabiliza a prática de testes automatizados como rede de proteção contra regressões (BECK, 2010; SOMMERVILLE, 2018). Pressman e Maxim (2016) lembram que o custo de corrigir um defeito cresce conforme ele avança no ciclo de vida, argumento que justifica o investimento em testes em um sistema já em produção.

2.3 Multiempresa e isolamento de dados

Um sistema entregue como serviço (SaaS) atende várias empresas na mesma instalação. Entre as estratégias possíveis de multiempresa — um banco por cliente, um esquema por cliente ou um banco compartilhado com coluna identificadora do proprietário do dado —, a terceira é a mais econômica e a mais arriscada: uma única consulta que esqueça o filtro expõe dados de um cliente a outro (ELMASRI; NAVATHE, 2019). O risco não é apenas técnico. A Lei Geral de Proteção de Dados Pessoais atribui ao controlador o dever de adotar medidas de segurança aptas a proteger os dados pessoais de acessos não autorizados (BRASIL, 2018).

O PostgreSQL oferece um mecanismo que desloca essa verificação do código da aplicação para o próprio banco: a segurança em nível de linha (Row Level Security — RLS), em que políticas declaradas na tabela determinam quais linhas cada sessão pode ver ou alterar, a partir de informações presentes na credencial apresentada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024). A diferença é relevante: com políticas ativas, uma consulta que esqueça o filtro não devolve dados alheios, devolve vazio. O serviço utilizado no projeto, o Supabase, expõe o banco por uma camada HTTP que transforma as declarações de um token assinado em variáveis legíveis pelas políticas, viabilizando esse arranjo (SUPABASE, 2025).

2.4 Interfaces conversacionais e modelos de linguagem com ferramentas

Modelos de linguagem de grande escala tornaram viável que um sistema seja operado por frases comuns, e não apenas por formulários. O padrão técnico que sustenta esse uso é a chamada de ferramentas (tool use): em vez de o modelo executar ações, ele recebe a descrição de funções disponíveis e devolve, de forma estruturada, qual função deveria ser chamada e com quais argumentos; quem executa é a aplicação (ANTHROPIC, 2025). A distinção é decisiva para sistemas que lidam com valores patrimoniais, porque modelos de linguagem são probabilísticos e podem produzir afirmações incorretas com aparência de correção.

Dessa limitação derivou a regra de divisão de trabalho adotada no projeto: o modelo interpreta a mensagem e redige a proposta; o código confere permissões, calcula o saldo, exige confirmação explícita do operador e grava. Nenhuma movimentação depende da aritmética do modelo. Trata-se, em termos de usabilidade, de preservar a visibilidade do estado do sistema e o controle do usuário sobre a ação, princípios clássicos de interação (KRUG, 2014), em um canal — o aplicativo de

mensagens — que a equipe de operação já domina e que não exige instalação nem treinamento de interface.

3 METODOLOGIA

3.1 Caracterização do trabalho e ferramentas utilizadas

O trabalho caracteriza-se como pesquisa aplicada, de natureza tecnológica, conduzida sob a forma de desenvolvimento de produto em ambiente real de uso. Não houve ambiente de laboratório isolado na etapa inicial: as primeiras versões foram publicadas e usadas por uma organização real, condição que impôs à equipe uma restrição metodológica forte — toda alteração precisava preservar os dados já existentes e manter o sistema disponível.

No lado do navegador, utilizaram-se React 19 com TypeScript, empacotados por Vite 6, estilização com Tailwind CSS 4, gerenciamento de estado com Zustand e gráficos com Recharts. No servidor, Node.js com Express, publicado em funções sem servidor na plataforma Vercel. A persistência é um banco PostgreSQL gerenciado pelo Supabase, acessado por sua biblioteca cliente. Os testes automatizados usam Vitest; o monitoramento de erros em produção, Sentry; a cobrança de assinaturas, a API do Mercado Pago; a integração com o WhatsApp, a Evolution API; e a interpretação de linguagem natural, a API do Claude, da Anthropic. O versionamento é feito em Git, com repositório no GitHub. A escolha do TypeScript em toda a pilha permitiu que as mesmas definições de tipo e as mesmas funções de regra de negócio fossem compartilhadas entre navegador e servidor, eliminando a duplicação da regra de saldo mínimo em dois idiomas.

3.2 Evolução do banco de dados por migrações numeradas

Toda alteração de estrutura do banco foi escrita como um arquivo SQL numerado e versionado junto ao código, com um arquivo de reversão quando a mudança era arriscada. Ao final do período relatado, o repositório reúne 57 migrações e 6 arquivos de reversão, totalizando cerca de 5.500 linhas de SQL. Cada arquivo começa por um comentário que explica o problema que ele resolve, o que acontece se for executado fora de ordem e qual migração é seu pré-requisito — documentação que se mostrou mais útil que qualquer registro externo, porque acompanha a mudança.

A migração central do projeto — a substituição do saldo em colunas pelo saldo por depósito — foi executada em quatro passos, seguindo a estratégia de expansão e contração, na qual o modelo antigo e o novo coexistem durante a transição. No primeiro passo, criaram-se as tabelas de unidades, de depósitos e de saldo por depósito, e preencheu-se a nova tabela a partir das colunas antigas. No segundo, instalou-se um gatilho no banco que mantinha as duas representações sincronizadas, de modo que o código antigo, ainda em produção, continuasse funcionando. No terceiro, com o servidor novo publicado, o gatilho foi removido e a gravação do saldo passou a ser responsabilidade de uma única função da aplicação. No quarto, as colunas antigas foram eliminadas.

A ordem desses passos não é indiferente, e a migração do terceiro passo registra explicitamente o motivo: o gatilho havia sido escrito para interromper-se de propósito caso existisse um segundo depósito central — para não escolher no chute em qual deles gravar. Isso significa que criar a primeira unidade nova, objetivo de toda a reestruturação, travaria as entradas e os consumos do sistema se o gatilho ainda estivesse ativo. A migração, por isso, verifica antes de agir: se as movimentações mais recentes não registram depósito de origem ou destino, quem as gravou foi o código antigo, o servidor novo não está no ar e a migração recusa-se a executar.

3.3 A regra de saldo como função pura

A decisão de projeto mais consequente foi concentrar em um único módulo tudo que lê ou grava saldo, e dentro dele separar o cálculo da gravação. A função de cálculo recebe os saldos do material, a operação pretendida, a quantidade e uma forma de obter o nome de cada depósito — para que a mensagem de recusa possa dizer onde faltou — e devolve uma de duas respostas: a recusa, com o texto que será exibido à pessoa, ou os saldos resultantes. Não há acesso a banco nem a rede nessa função.

A função devolve, junto com os saldos resultantes, a lista dos depósitos que a operação efetivamente movimentou. Essa distinção, que parece um detalhe, é o que impede a gravação de regravar saldos que a operação não tocou — e a seção 4.6 relata o que aconteceu antes de ela existir. Padronizou-se também o arredondamento em duas casas decimais na aplicação, e não apenas no banco, para que o servidor não decida "há saldo suficiente" olhando um número que o banco guardará diferente.

3.4 Permissões, isolamento e segurança

O isolamento entre empresas foi implementado em duas camadas. Na primeira, o acesso ao banco só é possível por uma camada que recebe a identificação da empresa na construção do cliente e aplica o filtro automaticamente nas tabelas pertencentes a uma organização, listadas explicitamente no módulo. A motivação está documentada no próprio código: com cerca de cem pontos de consulta no servidor, depender de cada um lembrar de escrever o filtro é o que vaza dado de um cliente para outro, porque basta uma consulta nova esquecer e ninguém percebe até alguém telefonar. Na segunda camada, quando a variável de ambiente com o segredo de assinatura está configurada, o servidor assina um token contendo o identificador da organização e passa a usar a chave anônima do banco: as políticas de RLS entram em vigor e o banco torna-se o fiscal. Alternar entre os dois modos é definir ou não uma variável de ambiente — nenhuma linha de código muda —, e em produção o segundo modo está ativo.

Dentro de cada empresa, as permissões combinam cargo e atribuição por depósito. O cargo define o que a pessoa pode fazer em geral — somente perfis administrativos registram entrada de material, por exemplo —, e a atribuição define em quais depósitos ela opera ou apenas consulta. Administradores não têm atribuição: veem e operam tudo da empresa. Complementarmente, aplicaram-se cabeçalhos de segurança HTTP com Helmet, limitação de taxa por endereço e por conta nas rotas sensíveis — login, criação de conta, recuperação de senha e pedido de demonstração —, armazenamento de senhas com bcrypt e autenticação por token assinado.

3.5 O assistente no WhatsApp: divisão de trabalho entre código e inteligência artificial

A operação pelo WhatsApp funciona por recepção de eventos: uma mensagem chega de um número, e o número identifica a pessoa e a empresa a que ela pertence. A partir daí o assistente age como aquela pessoa — vê os depósitos que ela vê, opera os que ela opera e apenas as operações liberadas para o canal. Ajuste de saldo, recontagem e cadastro permanecem exclusivos do painel, por serem operações de correção e não de rotina.

A divisão de trabalho é rígida. Consultas são respondidas de imediato, com dados lidos do banco. Gravações, não: a mensagem interpretada é convertida em uma proposta, apresentada à pessoa em texto — material, quantidade, depósito de origem e destino — e só executada depois de uma confirmação explícita, que pode exigir um código pessoal de quatro a seis dígitos quando a empresa configura essa exigência. Quem executa a movimentação é código, nunca o modelo. O mesmo desenho foi aplicado às cotações a fornecedores: o código redige a mensagem, envia, calcula o total e

fecha o pedido; o modelo faz apenas o que o código não faz, isto é, transformar uma resposta em linguagem natural como "o cimento está 32 e a tinta 180, entrego quinta" em preço por item. O modelo não negocia e não compra.

3.6 Testes, verificação e ambiente de homologação

A verificação apoiou-se em testes automatizados executados a cada alteração, abrangendo o cálculo de movimentação, as permissões por depósito, o isolamento entre organizações, a situação da conta e do plano, a interpretação de confirmações recebidas pelo WhatsApp, a alocação de lotes por validade, as transições de estado do patrimônio e a geração de arquivos de exportação. Incluiu-se também um teste de entradas aleatórias (fuzzing) sobre as funções de validação, para verificar que dados malformados produzem recusa e não exceção não tratada. Além disso, cada defeito relatado em produção passou a originar um teste de regressão escrito antes da correção, documentando no próprio teste o sintoma observado e a causa encontrada. Por fim, documentou-se e implantou-se um ambiente de homologação — segundo banco, segunda publicação, mesmo código — para que alterações de risco deixem de ser verificadas diretamente em produção.

4 RESULTADOS E DISCUSSÕES

4.1 O sistema entregue

O Stok Aí está em operação, publicado na internet e em uso por organizações reais. A parte do repositório correspondente ao produto reúne 174 arquivos TypeScript, com aproximadamente 32.700 linhas de código, além das 57 migrações de banco já mencionadas. A camada HTTP do painel expõe 87 rotas. A suíte de testes conta 635 casos em 53 arquivos, todos aprovados na última execução, com duração total inferior a cinco segundos — tempo que importa, porque uma suíte lenta deixa de ser executada.

Funcionalmente, o sistema oferece: cadastro de unidades, estoques, categorias e materiais, com unidade de medida, fator de conversão de volumes e estoque mínimo; registro de entrada, saída, consumo, transferência e ajuste, cada movimentação com responsável, data e justificativa; painel com indicadores de entrada, saída e movimentação interna por período, comparados ao período anterior; alertas de material abaixo do mínimo, na tela e por e-mail; auditoria com filtros por período, operação, material e responsável, exportável em PDF; exportação de materiais, movimentações, empréstimos, usuários, fornecedores e cotações em CSV compatível com planilhas em português; recontagem de inventário; gestão de usuários com cargos e atribuições por depósito; identidade visual da empresa (logotipo e cor) aplicada às telas; sistema de chamados de suporte; e assinatura com cobrança recorrente.

4.2 Os cinco modos de estoque

O resultado mais direto da reestruturação do modelo de dados foi deixar de existir um único tipo de estoque. Um estoque passou a ter nome livre, pertencer a uma unidade e ter um modo de operação, escolhido no cadastro, que determina quais telas e quais regras valem para ele. O Quadro 1 resume os cinco modos implementados.

Quadro 1 — Modos de operação de um estoque no Stok Aí

Modo	**O que controla e como opera**
Almoxarifado	Modo geral: entrada, saída, consumo, ajuste e transferência entre estoques da mesma unidade. O cadastro permite ligar apenas um subconjunto dessas operações.
Empréstimo	Retirada por tempo limitado, com devolução. Cada retirada registra tomador, quantidade e prazo, com prazo padrão em horas definido no cadastro do estoque.
Entrega	Entrega definitiva a uma pessoa, como equipamento de proteção individual e uniforme. Pode exigir assinatura desenhada na tela e controla a validade da troca por material.
Validade	Saldo por lote, com data de vencimento. A saída consome primeiro o lote que vence antes (FEFO), e o estoque avisa com a antecedência em dias configurada.
Patrimônio	Cada unidade física é um item com código próprio, número de série, estado (disponível, em uso, manutenção, baixado), responsável e histórico de eventos.

Fonte: elaborado pelos autores (2026).

A consequência prática é que a mesma instalação atende a necessidades que antes exigiriam sistemas diferentes: o controle de insumos de obra, o controle de ferramentas emprestadas, o controle de entrega de equipamentos de proteção individual, o controle de materiais perecíveis e o inventário de bens. E, o que mais importa para o objetivo do projeto, criar qualquer um desses estoques é preencher um formulário — não é alterar programa.

4.3 Estoque mínimo, ruptura e transferência

A regra de situação do material produziu um resultado que não estava previsto no início e que se mostrou o mais útil na rotina. Ao distinguir "zerado aqui, mas disponível em outro estoque da mesma unidade" de "zerado em toda a unidade", o sistema passou a separar dois pedidos que antes chegavam iguais ao setor de compras: o que exige comprar e o que exige apenas transferir. Como o estoque mínimo é o ponto de recompra da unidade, e não de cada depósito, o alerta de compra deixou de disparar por material que estava apenas no depósito errado. Materiais cadastrados com mínimo igual a zero são tratados como sem controle de mínimo: só alertam quando zeram em toda a unidade.

4.4 Recontagem: registrar que foi conferido, não só a divergência

A funcionalidade de recontagem expôs uma falha conceitual que vale relatar, por ser o tipo de erro que só aparece no uso. A primeira versão registrava a contagem física apenas quando ela discordava do sistema: a divergência virava um ajuste no histórico de movimentações. Quando a contagem batia, nada era gravado — a tela marcava o item como conferido apenas no armazenamento local do aparelho de quem contou. O efeito é que material esquecido e material conferido ficavam idênticos no banco, e o sistema não tinha como responder à pergunta mais operacional do inventário rotativo: o que ainda falta conferir?

A correção foi registrar o carimbo da conferência — quando passou por ele e a matrícula de quem contou — na própria linha de saldo, isto é, por material em um depósito, porque o mesmo material pode estar conferido no almoxarifado central e esquecido no de obra. A migração correspondente documenta que, na primeira execução, tudo apareceria como não recontado, e que isso é verdadeiro: o sistema não tem como saber o que foi conferido antes de a informação existir. Assumir a lacuna explicitamente mostrou-se preferível a preenchê-la por suposição.

4.5 Operação pelo WhatsApp e cotações a fornecedores

A operação pelo canal de mensagens atendeu ao objetivo de reduzir o atrito de registro. O almoxarife que antes anotava a saída em um papel para digitar depois — e frequentemente não digitava — passou a escrever a movimentação em uma frase, conferir a proposta e confirmar. Consultas de saldo são respondidas sem abrir o sistema. Como o assistente age com as permissões da pessoa que escreveu, nenhuma regra de acesso é relaxada para o canal, e o registro resultante é indistinguível, no histórico, de uma movimentação feita pelo painel: mesma movimentação, mesmo responsável, mesma auditoria.

A extensão desse mecanismo às cotações produziu o resultado mais inesperado do projeto. Fornecedores não precisam de cadastro, aplicativo ou portal: recebem a pergunta no WhatsApp e respondem como responderiam a um cliente, em texto livre. O sistema reconhece o número do fornecedor, associa a resposta à cotação aberta, extrai o preço por item e monta a comparação. O comprador vê um quadro comparativo construído a partir de conversas comuns. Aqui a inteligência artificial ocupa exatamente o espaço em que é insubstituível — interpretar texto livre — e nenhum espaço além dele: o total é calculado pelo código e o pedido é fechado por decisão humana.

4.6 Um defeito de concorrência e o que ele ensinou

O defeito mais instrutivo do período chegou como uma reclamação: um funcionário deu saída no almoxarifado de obra e o almoxarifado central foi descontado junto. A investigação mostrou que o cálculo estava correto — a função pura acertava os números em todos os cenários testados. O erro estava na gravação: o cálculo devolvia o mapa inteiro de saldos daquele material, incluindo os depósitos lidos no início da requisição e não movimentados, e a gravação escrevia linha por linha, inclusive as que a operação nunca tocou.

Enquanto nada mais acontecesse ao mesmo tempo, reescrever um valor igual ao que já estava lá passava despercebido. Mas entre a leitura e a gravação cabe outra operação: uma entrada registrada no central por outra pessoa era desfeita pela saída registrada na obra, que gravava por cima o saldo antigo do central. O estoque diminuía sem nenhuma movimentação que explicasse a diferença — e a narrativa do usuário, "descontou no central", era a descrição exata do sintoma, embora a causa não estivesse onde a frase sugeria.

A correção foi fazer o cálculo devolver, além dos saldos resultantes, a lista dos depósitos efetivamente movimentados, e a gravação escrever somente essas linhas. O caso ilustra três lições que a equipe incorporou ao método. A primeira é que cobertura de testes sobre a regra de negócio não garante correção do efeito: o cálculo estava testado e correto, o defeito morava na fronteira entre o cálculo e o banco. A segunda é que concorrência não é assunto apenas de sistemas de grande porte — bastam dois almoxarifes trabalhando ao mesmo tempo. A terceira é que o relato do usuário descreve o sintoma com precisão e a causa com imprecisão, e confundir os dois leva a corrigir o lugar errado. Defeitos semelhantes, originados na fronteira com a infraestrutura, apareceram na autenticação com o banco: tokens vencidos em instâncias mantidas quentes por tráfego contínuo, e tokens recusados por "emissão no futuro" quando o relógio do servidor estava meio minuto adiantado. Ambos foram resolvidos com renovação antecipada e com emissão datada dois minutos no passado — e ambos estão documentados em comentário no código, para que a próxima pessoa não os reintroduza.

4.7 Sustentabilidade do produto

Para que o sistema continue existindo depois do período letivo, implementou-se o modelo de assinatura em três planos — Essencial, Profissional e Empresarial —, com sete dias de avaliação gratuita, cobrança por cartão, Pix ou boleto, lembretes de vencimento por e-mail e um período de

carência antes do bloqueio. Os planos não se distinguem por propaganda, mas por limites e recursos lidos de um único módulo do código: número de unidades, de estoques por unidade e de usuários; e disponibilidade de transferência entre estoques, identidade visual, exportações, alertas por e-mail, operação por WhatsApp e cotações. Concentrar essa definição em um lugar só evitou a divergência comum entre o que a página de preços promete e o que o sistema efetivamente libera.

4.8 Limitações e dificuldades encontradas

O trabalho tem limitações que convém declarar. A principal é a ausência, durante a maior parte do período, de ambiente de homologação: as verificações ocorriam em produção, com dados reais, prática que a equipe reconhece como inadequada e que motivou a documentação e a implantação do segundo ambiente ao final do ciclo. A segunda limitação é a dependência de uma integração não oficial com o WhatsApp, sujeita a interrupções fora do controle do projeto; mitigou-se o risco com diagnóstico de conexão e avisos quando a instância para de receber mensagens, mas a dependência permanece. A terceira é o custo variável da interpretação por modelo de linguagem, enfrentado com limite mensal de mensagens por organização e com a regra de só acionar o modelo quando o código não resolve. A quarta é que as migrações são aplicadas manualmente no console do banco, sem automação de integração contínua, o que deixa a ordem de execução sob responsabilidade humana. A quinta é que os testes concentram-se nas regras de servidor e nas funções compartilhadas; a interface tem cobertura automatizada reduzida e é verificada manualmente. Por fim, não houve avaliação formal de usabilidade com instrumento validado nem medição quantitativa de ganho operacional nas empresas usuárias — a avaliação disponível é qualitativa, baseada em relatos de uso e em chamados abertos.

5 CONSIDERAÇÕES FINAIS

O objetivo central do projeto foi alcançado: o Stok Aí deixou de ser um controle com dois depósitos escritos no código e passou a ser um sistema em que unidade, estoque e modo de operação são cadastro da empresa. A transição ocorreu com o sistema em uso, sem perda de histórico e sem interrupção, por meio de migrações numeradas que explicitam, cada uma, o que resolvem e o que acontece se forem executadas fora de ordem. Sobre essa base estruturaram-se os cinco modos de estoque, o isolamento entre empresas fiscalizado pelo banco, a operação pelo WhatsApp com confirmação explícita e as cotações a fornecedores.

Do ponto de vista formativo, três aprendizados se destacam. O primeiro é que, em software em uso, a ordem das mudanças é parte do projeto: a mesma alteração, aplicada em sequência diferente, deixa de ser melhoria e passa a ser incidente. O segundo é que separar o que decide do que grava — regra pura no centro, efeitos na borda — não é formalismo acadêmico: foi o que permitiu testar a regra de saldo isoladamente e, no episódio relatado na seção 4.6, localizar um defeito que não estava na regra. O terceiro é que inteligência artificial em sistema de gestão rende mais quando se delimita o que ela não faz: interpretar linguagem, sim; decidir, calcular e gravar, não.

Como encaminhamentos para projetos futuros, a equipe aponta: automatizar a execução das migrações e os testes em integração contínua, eliminando a etapa manual; ampliar a cobertura automatizada da interface; incorporar leitura de código de barras e de QR na recontagem e na movimentação, reduzindo digitação no canteiro de obras; implementar funcionamento parcial sem conexão, situação frequente em obras; aplicar classificação ABC aos materiais, para que o esforço de controle se concentre nos itens de maior valor; e conduzir uma avaliação de usabilidade e de ganho operacional com instrumento formal, de modo a substituir a avaliação qualitativa atual por medida comparável.

REFERÊNCIAS

ANTHROPIC. Tool use with Claude. Anthropic API Documentation, 2025. Disponível em: https://docs.anthropic.com/en/docs/agents-and-tools/tool-use. Acesso em: 10 nov. 2026.

BALLOU, Ronald H. Gerenciamento da cadeia de suprimentos: logística empresarial. 5. ed. Porto Alegre: Bookman, 2006.

BECK, Kent. TDD: desenvolvimento guiado por testes. Porto Alegre: Bookman, 2010.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 10 nov. 2026.

CHOPRA, Sunil; MEINDL, Peter. Gestão da cadeia de suprimentos: estratégia, planejamento e operação. 6. ed. São Paulo: Pearson, 2016.

DIAS, Marco Aurélio P. Administração de materiais: uma abordagem logística. 6. ed. São Paulo: Atlas, 2019.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2019.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2003.

KRUG, Steve. Não me faça pensar: uma abordagem de bom senso à usabilidade web e mobile. 3. ed. Rio de Janeiro: Alta Books, 2014.

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation: row security policies. 2024. Disponível em: https://www.postgresql.org/docs/current/ddl-rowsecurity.html. Acesso em: 10 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.

SUPABASE. Row Level Security. Supabase Documentation, 2025. Disponível em: https://supabase.com/docs/guides/database/postgres/row-level-security. Acesso em: 10 nov. 2026.